Linked List - 2

Agenda:

1. Reverse the Linked List
2. Reverse the Linked List using Recursion
3. Shallow Copy vs Deep Copy
4. Deep Copy of linked list
5. Middle Element of List

① Reverse a Linked List (Iterative)

Logic: we use 3 pointers (prev) (initially null), (curr) (initially head), and (next) (to store the next node before breaking the link). In each step, we make the current node point backward to prev, then move all pointers one step forward.

T.C = O(N)
S.C = O(1)

Code →

```
public Node reverseIterative (Node head){
    Node prev = null;
    Node curr = head;
    Node next = null;
    while (curr != null){
        next = curr.next;      // Store next node
        curr.next = prev;      // reverse the link
        prev = curr;           // Move prev forward
        curr = next;           // Move curr forward.
    }
    return prev;               // New head of reversed List
}
```

② Reverse a Linked List (Recursive)

↳ uses call stack to reach the end of list and then reverse the links as the recursion unwinds

T.C = O(N)
S.C = O(N)

Logic: 1. Base Case: If the list is empty or has only one node, return that node.

2. Recursive Step: Recursively call the function for head.next. This returns the new head of the reversed portion

3. Link Reversal: Make the next node's (next) pointer point back to the current node (head.next.next = head) and set the current node's (next) to null.

code →

```
public Node reverseRecursive (Node head) {
    if (head == null || head.next == null) { return head};
    Node newHead = reverseRecursive (head.next);
    head.next.next = head;      // Poin next node back to current
    head.next = null;           // Break original fwd. link.
    return newHead;
}
```

Shallow vs Deep Copy

Feature	Shallow Copy	Deep Copy
• Nodes created →	0 (only references copied)	New nodes created for all elements
• Memory →	Shared (both point to same list)	Independent (Seperate lists)
• Saf to Modify →	No (changing one affects both)	Yes (changes are independent)
• Complexity →	O(1)	O(N)

---

③ # Deep Copy of a Linked List

Logic: $\boxed{\text{First Pass}}$: Create a new node for every original node and store them in the map : map.put (ON, NN)

$\boxed{\text{2nd Pass}}$: Set the next & random pointers for the new nodes by looking up the original node's pointers in the map.

code →
```
public Node copyRandomList (Node head) {
    if (head == null) return null;

    // Map: Key = Original Node, Value = Copied Node
    HashMap <Node, Node> hm = new HashMap <>();

    // Step-1: Create all new nodes and store in map
    Node curr = head;
    while (curr != null) {
        hm.put (curr, new Node (curr.val));
        curr = curr.next;
    }

    // Step-2: Set the pointers for the new nodes
    curr = head;
    while (curr != null) {
        Node newNode = hm.get(curr);
        newNode.next = hm.get(curr.next);
        newNode.random = hm.get(curr.random);
        curr = curr.next;
    }
    return hm.get(head)
```

(4) Middle Element of a Linked List

- Logic (Fast & Slow Pointers):

Use 2 pointers starting at the head. The slow pointer moves 1 step at a time, while the fast pointer moves 2 steps. When fast or fast.next becomes null, slow will be at the middle.

```
public Node findMiddle (Node head) {
    if (head == null) return null;
    Node slow = head;
    Node fast = head;

    # fast moves twice as fast as slow.
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    return slow; // This is the middle node
}
```

- even → $\left(\frac{n}{2}+1\right)$ second middle
- odd → middle